

ICLR 2024 | Stop Proving Expressiveness After the Fact, Build It In

G2N2, from the LITIS and LIFAT labs in France, turns a matrix-language fragment into a provably 3-WL graph neural network by way of a context-free grammar

The standard ruler for how much a graph neural network (GNN) can tell apart is the Weisfeiler-Lehman (WL) hierarchy: if two non-isomorphic graphs already look identical to the k -th-order WL test, no GNN in that class will separate them either. For years the field has worked in one direction only. You design a model on intuition, then go back and prove it matches, say, the third-order test. The expressiveness guarantee always arrives afterward, as a separate proof stapled onto the architecture.

The trouble is that there is no general recipe for that. Every new architecture needs its own bespoke expressiveness proof, and the reverse direction, starting from a mathematical object whose power you already know and mechanically building a network that inherits it, has been missing. Existing 3-WL models each pay a price: 3-IGN's operator basis grows like the 2k-th Bell number and is memory-hungry, while PPGN is tractable but was only shown to mimic 2-FWL, with no constructive bridge from a language to the model.

This ICLR 2024 paper, G2N2, runs the process the other way. Instead of "design first, prove later," it picks a language fragment whose expressiveness is already known, then reads the network's layers straight off that fragment's rules. The whole pipeline is automatic, the resulting network is provably 3-WL, and in practice it turns out faster and more accurate than the existing 3-WL models. The figure below is the whole paper in one picture.

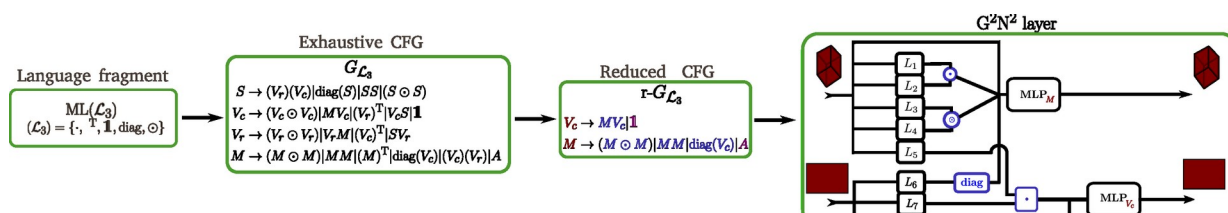


Figure 1. The full G2N2 derivation pipeline. Starting from the matrix-language fragment $ML(L3)$, which is equivalent in power to the 3-WL test, the authors write an exhaustive context-free grammar $G(L3)$, reduce it to the compact $r-G(L3)$, and then read a G2N2 layer directly off the reduced grammar: surviving variables become the network's inputs and production rules become the layer's operations.

Paper: <https://arxiv.org/abs/2303.01590> Code: <https://github.com/JPiquenot/Weisfeiler-and-Lehman-go-grammatical>

Background: from the WL hierarchy to a matrix language

The idea grows from a clean observation: the 1-WL and 3-WL tests can each be rewritten as a fragment of a matrix language called MATLANG. Concretely, two graphs are indistinguishable to 3-WL exactly when every "sentence" you can write in the fragment $ML(L3)$

evaluates to the same value on both. That is a genuine bridge between combinatorics (the WL test) and linear algebra (matrix operations).

But a bridge is not yet a road. Turning such a fragment into an actual trainable network had only ever been done case by case, and the resulting models could rarely claim the full 3-WL guarantee. GNNML3, derived from the same MATLANG view, was shown to be more expressive than 1-WL but never proven to reach 3-WL, precisely because there was no systematic derivation. G2N2's goal is to pave that road once and for all.

Step one: from a language to a grammar, then to a reduced grammar

The derivation begins from a set of operations: $L3 = \{ \cdot \text{ (matrix product), } ^T \text{ (transpose), } \mathbf{1} \text{ (the all-ones vector), } \text{diag} \text{ (take the diagonal), } \odot \text{ (element-wise product)} \}$. The authors first write an exhaustive context-free grammar $G(L3)$ whose generated sentences are exactly the fragment $ML(L3)$. This step turns "a language" into "a set of production rules."

Then comes the key move: reduction. While proving at every step that no 3-WL expressiveness is lost, they strip away redundant rules and variables until only the essential core productions remain: the reduced grammar $r\text{-}G(L3)$. Reduction both makes the eventual network leaner and, as a bonus, reveals which operations actually matter. To see how a rule becomes a computation, look at one iteration of the procedure below.

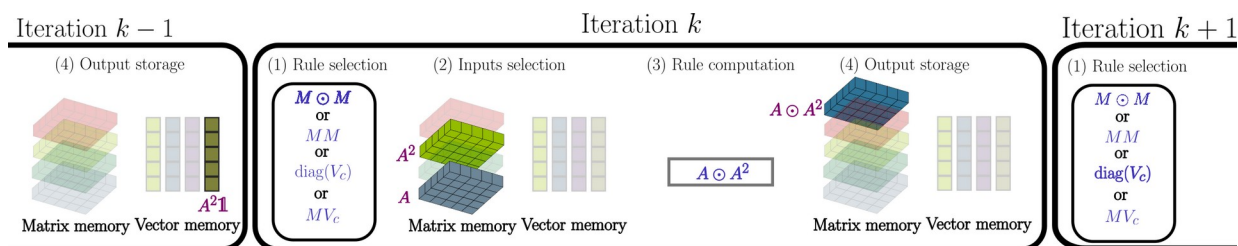


Figure 2. One iteration in four steps. (1) Rule selection: pick a production from $(M \odot M)$, (MM) , $\text{diag}(V_c)$, (MV_c) . (2) Input selection: fetch the operands that rule needs from the matrix memory and vector memory. (3) Rule computation: apply the operation (here, $A \odot A^2$). (4) Output storage: write the result back into the matching memory. Iterating this loop gradually accumulates richer graph features in memory.

Step two: read the grammar rules as network layers

Once the reduced grammar is in hand, the network is almost literally read off it: the surviving variables tell you what the inputs should be (a tensor of node and edge features), and each surviving production rule becomes a piece of a layer. That correspondence is where the name G2N2, Grammatical Graph Neural Network, comes from.

A layer maintains two memories: an edge (matrix) memory C and a node (vector) memory H . Learnable linear blocks $L1$ through $L7$ combine slices of these tensors, the reduced rules $(M \odot M)$, (MM) , $\text{diag}(V_c)$, and (MV_c) are computed on top of them, and two MLPs, MLP_M and MLP_Vc , concatenate and merge the pieces into the next layer's C and H . The figure below unpacks the data flow inside one layer.

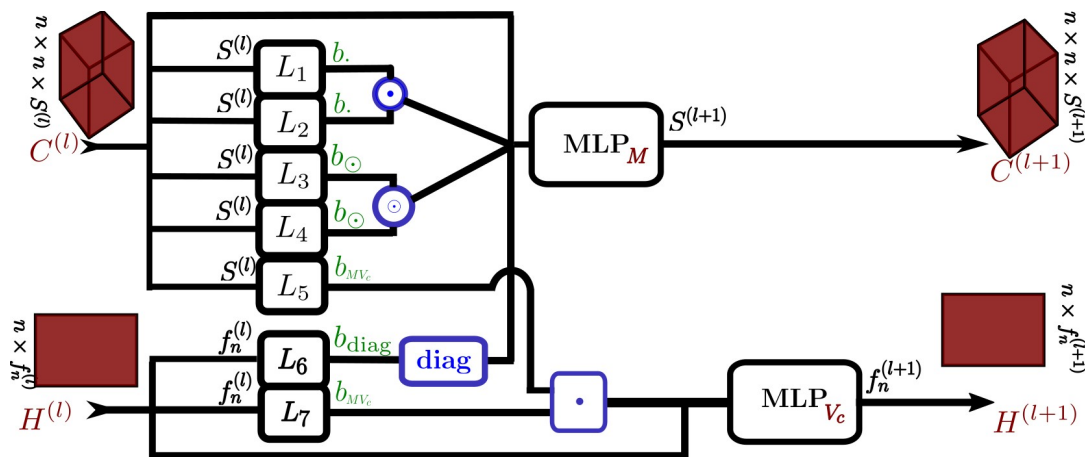


Figure 4. Inside a single G2N2 layer. The edge tensor C and node tensor H enter on the left; L_1 – L_5 linearly combine slices of C and take pairwise products following the rules (MM) and $(M \odot M)$, while L_6 – L_7 process H and turn it into diagonal matrices via diag . These intermediates, together with the inputs, feed MLP_M to produce the new edge tensor $C^{(l+1)}$ and MLP_{Vc} to produce the new node tensor $H^{(l+1)}$.

Stack k such layers, cap them with permutation-equivariant readout functions, and a complete G2N2 takes shape. The readouts act separately on the node memory H and on the diagonal and off-diagonal elements of the edge memory C , and a final decision layer turns their outputs into a prediction. The overall architecture looks like this.

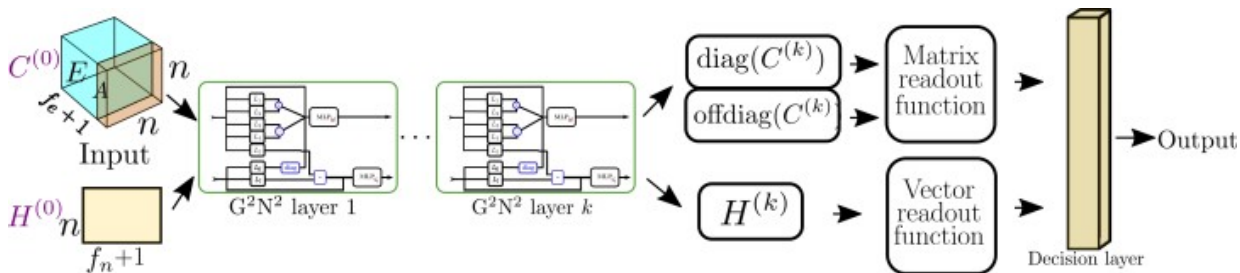


Figure 5. The full G2N2 architecture, graph to output. The input node and edge embeddings are updated by k stacked G2N2 layers; readout functions are then applied independently to $H^{(k)}$ and to the diagonal and off-diagonal parts of $C^{(k)}$, and a decision layer produces the final prediction.

Does it work? QM9 molecular regression

Beyond the theory, G2N2 holds up empirically. The first arena is QM9: roughly 130K small molecules with 12 quantum-chemical regression targets, of which R^2 is widely regarded as the hardest to predict. Trained one target at a time, G2N2 achieves the lowest MAE on all 12 targets while also running faster per epoch (98 seconds versus PPGN's 129). The most dramatic case is R^2 : the error drops from PPGN's 3.78 to 0.342, roughly an 11-fold improvement, and when all 12 targets are learned jointly, the gap widens further (16.07 versus 1.19).

Table 1. Selected QM9 single-target regression results (MAE, lower is better) plus mean per-epoch time. Full results for all 12 targets are in the paper's Table 4.

Target	PPGN	G2N2
μ	0.0934	0.0703

α	0.318	0.127
R^2	3.78	0.342
U0	0.022	0.0169
Cv	0.144	0.0429
Time / epoch	129 s	98 s

Classification and a spectral stress test

The second arena is the classic TUD graph-classification benchmark, spanning molecular graphs like MUTAG and PTC and social graphs like IMDB. G2N2 beats the second-best GNN competitor on five of the six datasets, and ranks first outright on MUTAG, PTC, and Proteins.

Table 2. TUD results comparing G2N2 with the strongest GNN competitor (accuracy, higher is better; G2N2's rank among GNNs in parentheses). Full results are in the paper's Table 6.

Dataset	G2N2	Rank	Best GNN competitor
MUTAG	92.5 $\pm$ 5.5	1	92.2 $\pm$ 7.5
PTC	72.3 $\pm$ 6.3	1	68.2 $\pm$ 7.2
Proteins	80.1 $\pm$ 3.7	1	77.4 $\pm$ 4.9
NCI1	82.8 $\pm$ 0.9	3	83.5 $\pm$ 2.0
IMDB-B	76.8 $\pm$ 2.8	2	77.8 $\pm$ 3.3
IMDB-M	54.0 $\pm$ 2.9	2	54.3 $\pm$ 3.3

The third arena is closer to a stress test: learning spectral filters on 900-node graphs. Message-passing networks are known to behave like low-pass filters by default, so high-pass and especially band-pass filtering is where expressiveness really shows. The result is telling. PPGN nearly maxes out low-pass and high-pass but collapses on band-pass (R^2 of just 0.1041), whereas G2N2 stays strong across all three, reaching a band-pass R^2 of 0.8206. The authors argue PPGN fails here precisely because it lacks the kind of explicit edge memory that G2N2 keeps.

Table 3. R^2 on spectral node regression (median of 10 runs, higher is better). PPGN fails on band-pass filtering; G2N2 remains robust across all three filter types.

Method	Low-pass	High-pass	Band-pass
PPGN	0.9991	0.9925	0.1041
G2N2	0.9996	0.9994	0.8206

Which rules matter: the grammar-reduction ablation

The experiment that best captures the method's flavor is the ablation around grammar reduction. On the QM9 R^2 target, the authors compare the full grammar G(L3), an intermediate grammar i-G(L3), and the reduced grammar r-G(L3): their errors are essentially the same, confirming that reduction only removes redundancy and leaves expressiveness untouched. But once you push past the reduced grammar and start deleting core rules, the error climbs noticeably.

That degradation is itself useful information: it quantifies how much each operation contributes to the final accuracy. In other words, when a task does not actually need the full 3-WL power, this plot tells you which rules you can safely drop, so you can prune the model deliberately instead of hacking away at parameters and hoping accuracy survives.

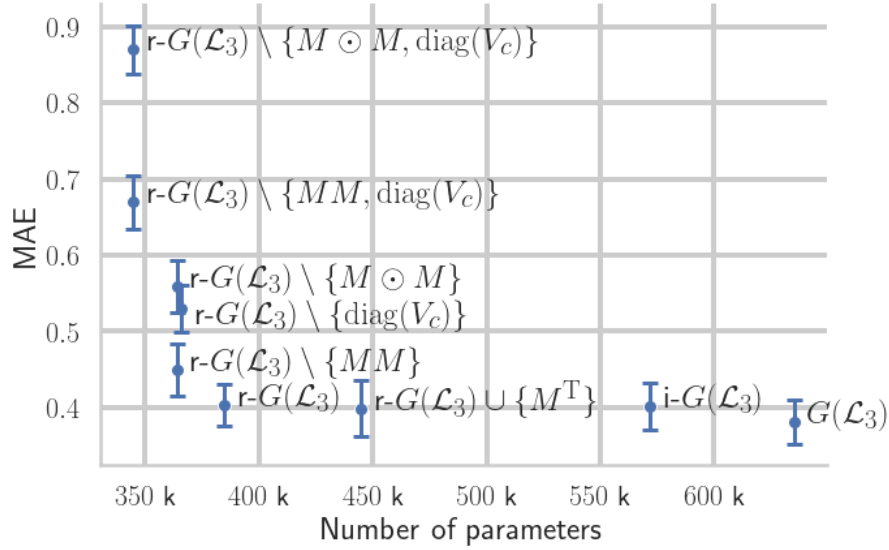


Figure 6. Parameter count versus MAE on QM9's R^2 target. The full grammar $G(\mathcal{L}_3)$, the intermediate $i-G(\mathcal{L}_3)$, the reduced $r-G(\mathcal{L}_3)$, and a variant adding one rule (M^T) all land at comparable error; removing core rules such as $\{MM\}$, $\{M \odot M\}$, or $\text{diag}(V_c)$ from $r-G(\mathcal{L}_3)$ (marked with $\setminus$) drives MAE up, confirming each rule pulls its weight.

Takeaway: expressiveness by construction

What G2N2 really changes is the workflow. Instead of designing a network and then hoping to prove it is expressive enough, you start from a language whose power you already know, reduce it to a clean grammar, and read the network directly off the rules, with expressiveness guaranteed by construction. The concrete payoff is a provably 3-WL model that is also faster and more accurate than its predecessors.

The more interesting point is that the "grammatical" framework is generic. In principle, any fragment of an algebraic language could be sent down the same route and turned into a network that inherits its expressive power. G2N2 is the first concrete instance of that idea, and it points toward a whole family of GNN designs where you choose the expressiveness you want up front and build the model to match.